第25课 字符串（一）—— char 数组与 string

一、课程信息

项目	内容
课次	第 25 课 / 共 40 课
所属阶段	阶段三：数据结构与进阶（第 1 课）
主题	字符串（一）—— char 数组与 string
前置知识	阶段一全部语法 + 阶段二基础算法（含数组、循环、函数、排序等）
课时	100 分钟

教学目标（可检验）

1. **能用 `string` 类型声明、赋值、输入输出字符串**，包括用 `cin` 和 `getline` 两种方式读入，并说出它们的区别。
2. **能用 `s.length()` 获取字符串长度，用 `s[i]` 访问指定位置的字符**，并在程序中正确使用。
3. **能独立完成"统计字符串中某类字符个数"类型的基础题目**（如统计元音字母个数）。

二、教学重点与难点

重点

- `string` 类型的声明、赋值、输入输出（`cin` vs `getline`）
- 用下标 `s[i]` 逐个访问字符串中的字符
- `s.length()` / `s.size()` 获取长度

难点

- 理解 `cin >> s` 遇空格停止 vs `getline(cin, s)` 读整行的区别
- 下标从 **0** 开始（而非 1），最后一个字符的下标是 `s.length() - 1`
- `cin` 与 `getline` 混用时的"换行符残留"问题（本课仅简单提及，不深入）

三、课堂时间分配（共 100 分钟）

时间	时长	环节	内容
0′–5′	5 min	开场导入	阶段三开篇 + 生活中的字符串
5′–25′	20 min	讲解一	字符串概念、char 数组（简提）、string 基础
25′–35′	10 min	例题 1–2	边讲边练：声明、赋值、输入输出
35′–55′	20 min	讲解二	cin vs getline、长度、下标访问
55′–70′	15 min	例题 3–4	边讲边练：长度、逐字符遍历
70′–90′	20 min	课堂练习	学生独立完成 2–3 题，教师巡视指导
90′–100′	10 min	总结 + 布置作业	回顾要点、常见错误、预告下节课

> **设计原则**：两段讲解各不超过 25 分钟，每段讲完立刻上手写代码，保证"讲一点、练一点"。

四、讲稿提纲

4.1 开场导入（5 分钟）

> 同学们，今天我们正式进入**阶段三**啦！ 前面两个阶段，我们学会了 C++ 的基本语法和好多有趣的算法，就像学会了拼音和查字典。从今天开始，我们要学习更多"工具"，让我们能解决更厉害的题目！
> 阶段三的第一个新朋友，叫做——**字符串**。

引出字符串的概念：

> 大家想一想，我们之前学过 `int` 存整数、`double` 存小数、`char` 存一个字符。可是——
> 如果我想让电脑记住你的**名字**呢？名字可不是一个字符，而是好几个字符拼在一起！比如"小明"、"Alice"……
> 像这样**一串字符排成一队**，就叫做**字符串**。
> 你可以把字符串想象成一条**手链**——手链上穿了好多颗**珠子**，每一颗珠子就是一个字符。整条手链就是一个字符串！

4.2 讲解一：字符串基础（20 分钟）

（1）char 数组——老办法（简单提及，约 3 分钟）

> 在很久以前的 C 语言里，人们用 **char 数组** 来存字符串，就像用一排小格子来放珠子：

```
char name[20] = "Alice";
```

> 这就像准备了 20 个小格子，把 A、l、i、c、e 一个个放进去。它能用，但是用起来不太方便——比如想知道长度要专门数，想拼接两个名字也很麻烦。
>
> 所以 C++ 给了我们一个**更好用的工具**—— `string`！以后在竞赛里我们**推荐用 string**，更方便、更不容易出错。

（2）string 类型——我们的新朋友（重点，约 17 分钟）

头文件：

> 要用 `string`，首先要在程序最上面 `#include<string>`。不过好消息是，很多时候 `#include<bits/stdc++.h>` 已经帮我们包含了（竞赛中常用万能头文件），所以一般不用额外加。

声明和赋值：

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    string s1;              // 声明一个空字符串，就像一条还没穿珠子的手链
    string s2 = "Hello";    // 声明并直接赋值
    string s3 = "World";
    cout << s2 << endl;     // 输出：Hello
    return 0;
}
```

> 看，是不是跟我们用 `int a = 5;` 很像？只不过 `int` 后面跟数字，`string` 后面跟用双引号括起来的文字。

输入和输出：

> **用 `cin` 输入**——最常用：

```
string name;
cin >> name;      // 键盘输入一个字符串（遇空格就停！）
cout << name;     // 输出
```

> 重点来了！ `cin` 读字符串有一个小脾气：**它看到空格就不读了！**
> 比如你输入 `Hello World`， `cin >> name` 只会读到 `Hello`， `World` 就被丢下了。
> 就像你让小助手去排队买冰淇淋，但他一看到有人（空格）就停下来不排了
> **用 `getline` 读整行**——空格也能读进来：

```
string sentence;
getline(cin, sentence);   // 读一整行，空格也照收!
cout << sentence;
```

> `getline` 就厉害了，它会**把一整行都读进来**，空格、标点什么的都不怕。就像一个很勤快的小助手，不管队伍多长都排到底

什么时候用谁？

场景	用哪个
读一个单词（没有空格）	`cin >> s`
读一整句话（有空格）	`getline(cin, s)`

> 竞赛题目中大多数情况用 `cin` 就够了。如果题目说"输入一行包含空格的字符串"，就用 `getline`。

4.3 例题 1–2 实操（10 分钟）

> （见第五板块"课堂例题"部分）

4.4 讲解二：长度和下标（20 分钟）

（1）字符串长度

> 想知道手链上有几颗珠子？用 `.length()` 或 `.size()`：

```
string s = "Hello";
cout << s.length() << endl;  // 输出 5
cout << s.size() << endl;    // 也输出 5，两个完全一样
```

> `.length()` 和 `.size()` 就像一个人的大名和小名，叫哪个都行，结果完全一样。老师习惯用 `.size()`，你们随意~

（2）用下标访问单个字符

> 手链上每颗珠子都有一个编号，**从 0 开始**！

```
字符串:  H  e  l  l  o
下标:    0  1  2  3  4
```

> 注意！不是从 1 开始，是从 **0** 开始！就像电梯的楼层，有的电梯是从 0 层开始的。
>
> 这个和我们之前学数组下标从 0 开始是一样的道理！

```
string s = "Hello";
cout << s[0] << endl;  // H —— 第一个字符
cout << s[1] << endl;  // e —— 第二个字符
cout << s[4] << endl;  // o —— 第五个字符（最后一个）
```

> 最后一个字符的下标是 `s.length() - 1`，不是 `s.length()`！

（3）用循环遍历字符串

> 我们可以用 `for` 循环把手链上的每颗珠子都看一遍：

```
string s = "Hello";
for(int i = 0; i < s.size(); i++){
    cout << s[i] << " ";
}
// 输出: H e l l o
```

> 这跟我们之前遍历数组是不是一模一样？没错！**string 就像一个装着字符的数组**，但比数组更好用！

4.5 例题 3–4 实操（15 分钟）

> （见第五板块"课堂例题"部分）

4.6 课堂总结（含在最后 10 分钟内）

> 今天我们学了一个超好用的新工具 `string`，来复习一下：
>
> 1. **字符串** = 一串字符 = 手链上穿着的一排珠子
> 2. **声明**：`string s = "Hello";`
> 3. **输入**：`cin >> s`（遇空格停）、`getline(cin, s)`（读整行）
> 4. **长度**：`s.size()` 或 `s.length()`
> 5. **访问单个字符**：`s[0]`、`s[1]`……下标从 0 开始
> 6. **竞赛推荐用 `string`**，比 char 数组方便！

七、常见错误与应对

错误 1：忘记引头文件

```
//  错误：没有 #include<string>，也没有万能头文件
using namespace std;
int main(){
    string s = "Hello";  // 编译报错！
}
```

应对：提醒学生竞赛中统一用 `#include<bits/stdc++.h>` 万能头文件，就不会忘。

错误 2：用 cin 读含空格的字符串

```
string s;
cin >> s;
// 输入：Hello World
// s 里只有 "Hello"，"World" 丢了！
```

应对：告诉学生口诀——**"cin 怕空格，getline 不怕"**。如果题目说"一行字符串"且可能有空格，就用 `getline`。

错误 3：下标越界

```
string s = "Hello";  // 长度5，下标0~4
cout << s[5];         //  越界！s[5]不存在
```

应对：强调下标从 0 开始，最大是 `s.size() - 1`。可以类比：手链有 5 颗珠子，编号是 0、1、2、3、4，没有 5 号！

错误 4：把 `s.size()` 写成 `s.size`（忘记括号）

```
cout << s.size;   //  少了括号，编译报错或输出奇怪的东西
cout << s.size(); //  正确
```

应对：强调 `.size()` 是一个**函数调用**，函数后面一定要有小括号，哪怕括号里什么都不写。就像打电话，你得拨号（加括号）才能接通！

错误 5：混淆字符和字符串的引号

```
string s = 'H';   //  单引号是 char，不能直接赋给 string
string s = "H";   //  双引号才是字符串

char c = "H";     //  双引号是字符串，不能赋给 char
char c = 'H';     //  单引号才是单个字符
```

应对：口诀——**"单引号穿一个，双引号穿一串"**。单引号 `' '` 里只放一个字符，双引号 `" "` 里放字符串。

九、下节课预告

> 下节课是 **第 26 课：字符串（二）—— 常用操作**。
>
> 今天我们学会了创建字符串、读入字符串、看它有多长、看它每一个字符。但是！字符串能做的事远不止这些——
>
> - 想把两个字符串**拼**在一起？
> - 想**比较**两个字符串谁大谁小？
> - 想从一个长字符串里**截一段**出来？
> - 想在字符串里**查找**某个字符在哪里？
>
> 这些超酷的操作，我们下节课一起来学！记得做完今天的作业哦~

本教案适用于小学四年级 CSP-J 启蒙课程，第 25 课 / 共 40 课。